

CYSTATION

Rapport de projet

<https://github.com/FLG95/CYStation>

El Jarjini Hicham
Goupil Florian
Jamhour Nassim
Jean-Louis Nicolas
Denon Capovilla Alessio

Sommaire

I. Introduction et Contexte.....	2
1. Présentation générale du projet.....	2
2. Choix du thème du projet.....	3
3. Objectifs et Enjeux.....	4
4. Outils et technologies choisis.....	5
II. Répartition des tâches.....	6
1. Rôles et apports.....	6
2. Planification.....	6
3. Méthode de collaboration.....	7
III. Étapes réalisées.....	8
1. Planification et Conception.....	8
2. Phase de Réalisation.....	9
3. Résultats obtenus.....	10
IV. Conclusion et perspectives.....	11
1. Bilan global.....	11
2. Pistes d'amélioration.....	12

I. Introduction et Contexte

1. Présentation générale du projet

L'Internet des Objets connaît aujourd'hui une expansion fulgurante, transformant notre monde physique par l'intégration de capacités d'échange de données numériques. Que ce soit dans le domaine de l'agriculture ou de la gestion urbaine, l'interconnexion des dispositifs permet d'automatiser, de contrôler et d'optimiser les ressources à une échelle inédite. C'est dans ce cadre d'innovation technologique que s'inscrit notre projet de développement web. Bien que les applications de l'IoT soient souvent associées aux villes ou aux maisons intelligentes, notre groupe a fait le choix de transposer ces concepts dans un environnement beaucoup plus atypique et extrême : une station spatiale de type ISS. Ce choix a été motivé par notre fascination commune pour l'espace, un univers riche en mystères qui stimule l'imaginaire tout en posant des défis techniques colossaux. La problématique principale que nous adressons à travers cet environnement est la gestion centralisée d'un espace clos soumis à de hautes contraintes. Dans une station spatiale, la survie et le confort de l'équipage dépendent d'un équilibre précaire. Notre plateforme vise donc à répondre à la nécessité de surveiller en temps réel les ressources vitales, de faciliter la communication avec l'extérieur, et de sécuriser les accès aux différents modules, tout en offrant une interface centralisée pour prévenir le moindre dysfonctionnement.

Notre projet se concrétise sous la forme d'une plateforme numérique intelligente baptisée "CYStation". Il s'agit d'une application web responsive conçue comme le véritable tableau de bord central de notre vaisseau spatial. L'objectif de CYStation est de centraliser la collecte, la visualisation et la gestion des données issues de multiples objets connectés virtuels répartis dans les différents modules de la station. Concrètement, la plateforme gère des équipements de haute importance, loin des objets connectés traditionnels. Parmi les objets phares simulés, on retrouve des régulateurs d'oxygène, des générateurs d'énergie, des systèmes de communication radio et des radars spatiaux. La valeur ajoutée principale de notre solution réside dans sa capacité à gérer un espace clos soumis à de hautes conditions de survie. Contrairement à une maison intelligente où une panne de capteur engendre un simple inconfort, CYStation met en scène un environnement où la priorisation des alertes et l'optimisation des ressources sont vitales. La plateforme intègre également une dimension évolutive avec un système de points d'expérience, permettant aux utilisateurs de débloquer progressivement des fonctionnalités complexes au fur et à mesure de leur familiarisation avec le système.

Le périmètre de notre projet englobe le développement complet d'une application web fonctionnelle. Cela inclut la conception de l'interface utilisateur pour offrir une navigation intuitive et immersive, le développement de la logique serveur, et la mise en place d'une véritable base de données relationnelle. Le projet comprend également le développement d'un système robuste d'authentification et de gestion des rôles, ainsi qu'un mécanisme de gamification calculant les points d'expérience à chaque interaction.

La plateforme CYStation a été structurée pour s'adapter à une hiérarchie stricte d'utilisateurs, reproduisant la chaîne de commandement d'un équipage spatial. L'accès aux modules et aux objets est régi par l'accumulation de points d'expérience. Le visiteur ne possède aucun droit d'action sur la station mais peut consulter la partie publique du site, lire des articles d'information ou découvrir le descriptif de la mission. L'utilisateur simple débute avec 0 XP. Son accès est restreint aux fonctions de confort. Par exemple, il peut interagir avec la machine à café connectée, chaque action lui rapportant 10XP. Il a également la capacité de rédiger et publier des articles sur la plateforme. Dès qu'il atteint 50XP, il évolue au rang supérieur, débloquent l'accès à des équipements de surveillance comme le radar et la radio. Cet utilisateur a un rôle crucial dans le maintien des opérations. Il gère des éléments structurels vitaux tels que le générateur d'énergie et les réserves d'eau. L'Administrateur est accessible à partir de 100 XP. C'est les commandants de la station. Il a la main sur les paramètres les plus critiques à la survie de l'équipage : les niveaux de CO₂, l'oxygène et les radiations. Il dispose d'un panel d'administration exclusif lui permettant d'ajouter de nouveaux modules à la station et de gérer l'ensemble des utilisateurs.

2. Choix du thème du projet

L'idée de concevoir un système d'information pour une station spatiale est née de nos centres d'intérêt et de l'actualité scientifique. En effet, notre groupe partage une fascination pour l'univers, ses paysages cosmiques, l'observation des étoiles et l'immense part de mystère qui reste encore à découvrir. Cette appétence pour l'espace est d'ailleurs largement nourrie par la culture populaire et la littérature de science-fiction, à l'instar d'œuvres marquantes comme *Projet Dernière Chance*, qui illustrent avec brio la vulnérabilité et l'ingéniosité humaine face à l'immensité du vide spatial. Au-delà de l'imaginaire, c'est l'actualité aérospatiale, notamment avec les préparatifs de la mission Artemis II marquant le retour programmé de l'humanité vers la Lune. L'espace demeure un secteur en plein essor et largement inconnu, ce qui en fait un terrain parfait pour imaginer des solutions technologiques novatrices. Il nous a semblé particulièrement stimulant de transposer un sujet académique de développement web dans ce cadre atypique.

Déployer un système de l'Internet des Objets dans un milieu aussi extrême que l'espace revêt une pertinence aujourd'hui. L'aérospatial est un secteur émergent et stratégique où l'automatisation, la fiabilité des communications et la collecte de données en temps réel sont devenues primordiales. Contrairement à la domotique terrestre, où une défaillance de capteur n'engendre qu'un simple inconfort, l'IoT spatial se définit véritablement comme un IoT de survie. Dans ce contexte, la rigueur imposée est absolue : les systèmes doivent fonctionner de manière infaillible, sans latence fatale. L'erreur n'y est tout simplement pas permise. Concevoir une architecture logicielle capable de simuler cette exigence de haute fiabilité nous permet d'anticiper les défis technologiques de demain. À l'aube des futures missions habitées vers Mars, la gestion autonome des ressources vitales par des réseaux de capteurs interconnectés n'est plus une utopie lointaine, mais bien le prochain grand bond en avant de l'ingénierie logicielle. Ce projet s'inscrit pleinement dans cette dynamique.

D'un point de vue technique et professionnel, la réalisation de la plateforme CYStation représente un véritable défi de conception pour notre équipe. Notre principale motivation intellectuelle réside dans l'architecture d'un système complexe et interconnecté. Le développement du Back-End et la modélisation de la base de données nous stimulent particulièrement : l'objectif est de bâtir une infrastructure robuste où les données des différents modules interagissent de manière parfaitement fluide. Parallèlement, concevoir une interface utilisateur que nous voulons belle, immersive et futuriste constitue un défi créatif majeur. Sur le plan professionnel, ce projet est l'opportunité idéale d'explorer et d'appliquer concrètement des frameworks modernes de développement web. L'acquisition de ces compétences techniques pointues, alliée à la mise en œuvre de logiques métier originales, constitue un socle de connaissances solide que nous pourrions directement valoriser dans nos futures carrières d'ingénieurs.

3. Objectifs et Enjeux

L'objectif ultime et mesurable de notre projet est de concevoir, développer et livrer une application web totalement fonctionnelle et utilisable, déployée sur un environnement local. Bien que l'ambition d'un hébergement en ligne ait été envisagée, nous avons fait le choix stratégique de concentrer nos efforts sur la robustesse du code et la complétude des fonctionnalités requises par le cahier des charges, en garantissant une exécution locale sans faille. Ce livrable final doit se présenter comme une preuve de concept aboutie de la plateforme CYStation. Il doit permettre lors de la soutenance de simuler de bout en bout l'expérience utilisateur, de la simple visite jusqu'à l'administration complète de la station spatiale, avec une interface réactive et une base de données opérationnelle.

Pour atteindre notre objectif principal, le projet a été découpé en plusieurs jalons fonctionnels. Les grandes étapes clés à valider incluent la mise en place d'un système d'authentification et de connexion sécurisé, différenciant immédiatement les rôles. La création d'une architecture orientée objet pour structurer proprement les entités de notre station. L'automatisation de l'affichage dynamique sur l'interface Front-End pour que les données des capteurs se mettent à jour de façon cohérente. Le développement du panel d'administration, véritable tour de contrôle permettant la gestion des droits et des modules. Au-delà de ces objectifs fonctionnels, ce projet place un objectif d'apprentissage massif

pour notre équipe. Au lancement du projet, nous ne possédions aucune connaissance préalable sur les frameworks web ni sur la conception avancée de bases de données. L'objectif était donc d'assimiler et de maîtriser ces nouvelles technologies de zéro, en faisant preuve d'une grande capacité d'auto-formation dans un laps de temps extrêmement réduit.

Dès l'entame du projet, plusieurs obstacles majeurs ont été identifiés, nécessitant une gestion prudente. Le premier était la courbe d'apprentissage : devoir maîtriser un framework web savoir bien l'implémenter en partant de zéro, tout en respectant une deadline stricte, représentait le risque principal d'accumulation de retard. Ensuite, la gestion du temps s'est avérée critique : le volume de travail exigé par le cahier des charges confronté au temps imparti nous a obligés à faire des choix stricts et à éviter de nous disperser. Enfin, la coordination et le travail collaboratif posaient un défi : l'utilisation de Git et de la plateforme GitHub, impliquait un risque élevé de conflits de code ou d'écrasement de données.

4. Outils et technologies choisis

L'architecture globale de CYStation repose sur un modèle client-serveur classique. Elle se scinde en deux parties distinctes interagissant en temps réel : un Front-End conçu en HTML, CSS et enrichi par JavaScript et Thymeleaf pour structurer une interface immersive, et un Back-End chargé de centraliser la logique métier, l'authentification et les accès à la base de données contenant les informations vitales de la station spatiale.

Afin de répondre aux exigences du projet tout en maximisant notre apprentissage, nous avons sélectionné la pile technologique suivante. L'interface utilisateur a été conçue à l'aide des langages web standards (HTML et CSS) pour structurer et styliser l'affichage des composants de la station. La logique serveur, quant à elle, a été développée en Java en s'appuyant sur le framework Spring Boot. Enfin, la sécurité est renforcée par Spring Security. Les données étant au centre de l'expérience, nous avons opté pour le système de gestion de base de données relationnelle MariaDB. Ce choix permet de garantir l'intégrité référentielle entre les rôles des utilisateurs et les objets connectés. L'interaction avec la base MariaDB est abstraite par Spring Data JPA. La couche de persistance exploite des interfaces telles que DeviceRepository, qui hérite de JpaRepository pour obtenir nativement les méthodes CRUD : Create, Read, Update, Delete.

Le projet a été développé et optimisé pour un hébergement en local, en exploitant le serveur web intégré de Spring Boot et une instance locale de MariaDB. Ce choix garantit un environnement stable et maîtrisé pour la maintenance, exempt des problématiques de latence réseau externes.

II. Répartition des tâches

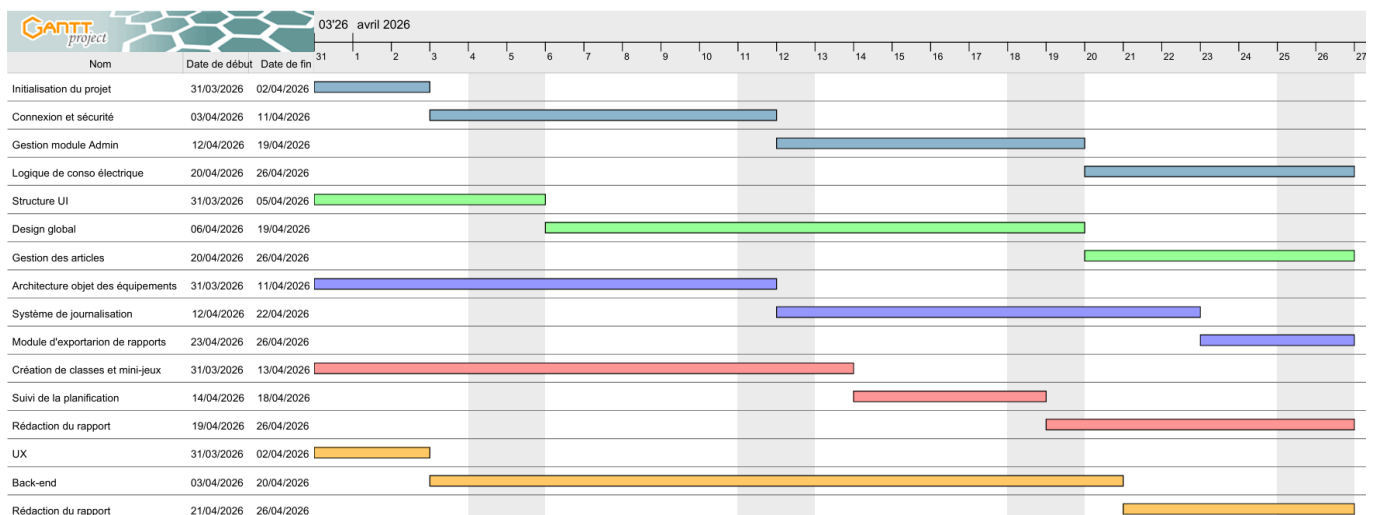
1. Rôles et apports

L'organisation s'est structurée autour des compétences techniques et préférences de chacun, permettant d'équilibrer la charge entre l'architecture, l'interface et la gestion documentaire.

- **Florian (Architecte Back-end & Sécurité)** : Initialisation du projet (Spring, MariaDB), configuration du système de connexion et de sécurité, développement du panneau d'administration et implémentation de la logique de consommation électrique systémique.
- **Hicham (Lead Front-end & UI)** : Conception du squelette de l'interface utilisateur, intégration du design global et développement de la logique Back-end de gestion des articles du site.
- **Nassim (Développeur Back-end)** : Modélisation de l'architecture objet des équipements (classe mère Device), mise en place du système de journalisation (logs) et création du module d'exportation de rapports téléchargeables.
- **Nicolas (Gestion de projet & Développeur)** : Suivi de la planification via l'élaboration du diagramme de Gantt, structuration du rapport et développement applicatif (implémentation d'une classe fille Device et création d'un mini-jeu intégré).
- **Alessio (Intégrateur & Développeur)** : Enrichissement de l'expérience utilisateur (intégration des avatars et ressources graphiques), participation à la rédaction et développement d'un équipement spécifique héritant de la classe Device.

2. Planification

La progression du projet a été dictée par des dépendances techniques strictes (jalons bloquants). L'initialisation de l'environnement serveur et de la base de données par Florian était un prérequis absolu pour l'intégration de l'interface par Hicham. Sur le plan de l'architecture logicielle, le développement des équipements spécifiques par Nicolas et Alessio dépendait directement de la finalisation préalable de la classe abstraite Device conçue par Nassim.



3. Méthode de collaboration

L'infrastructure technique a posé des défis d'intégration initiaux pour synchroniser cinq développeurs sur un même dépôt. La gestion des versions a entraîné des conflits fréquents, collisions lors des merges et rebases en raison d'interventions simultanées sur des fichiers communs.

Pour sécuriser l'intégration continue, la coordination s'est articulée autour de sessions vocales Discord quotidiennes. L'utilisation systématique du partage d'écran a permis de valider collégialement les modifications critiques de code avant chaque commit, évitant ainsi les écrasements asynchrones. La gestion documentaire et l'avancement théorique ont été centralisés sur Google Docs pour garantir un environnement de rédaction collaboratif et stable.

III. Étapes réalisées

1. Planification et Conception

La réussite d'un projet de développement web repose en grande partie sur la rigueur de sa phase d'initialisation. Face à la densité du cahier des charges, notre équipe a dû adopter une approche méthodique pour structurer le travail avant même d'écrire la première ligne de code. Le lancement du projet s'est matérialisé par une première réunion physique réunissant l'ensemble du groupe. L'objectif de cette séance de travail était de décortiquer en détail les attentes du cahier des charges fourni au format PDF. Pour centraliser nos réflexions et nos notes, nous avons utilisé un document collaboratif Google Doc, permettant à chacun de s'appropriier les différentes contraintes. Dès cette étape d'analyse, une priorisation a été définie. Nous avons acté que le cœur du système reposait sur la manipulation des données. Ainsi, la priorité absolue a été donnée à la conception du Back-End et à la visualisation de la base de données.

La phase de conception s'est divisée en deux axes majeurs : l'interface utilisateur et l'architecture logicielle. Afin de matérialiser l'univers de la CYStation, nous avons commencé par réaliser plusieurs croquis sur papier. Cette méthode, simple et rapide, nous a permis de visualiser la disposition des éléments clés du tableau de bord et de nous accorder sur l'expérience utilisateur globale avant de passer au développement HTML/CSS. Pour structurer notre base de données MariaDB, nous avons élaboré un schéma conceptuel initial. Bien que ce schéma ait naturellement évolué au fil du développement, il nous a fourni une fondation solide. Notre modèle s'articule autour de quatre tables :

- User : Gestion des profils, des mots de passe, des rôles (Passenger, Scientist, Admin) et des points d'expérience (XP).
- Device : Recensement des objets connectés (radars, générateurs, capteurs d'oxygène...) et de leurs états (actif/inactif).
- Zone : Représentation spatiale des différents modules de la station spatiale abritant les équipements.
- Article : Stockage des publications et informations rédigées par les utilisateurs ayant les droits requis.

Au lieu d'utiliser un outil de micro-gestion complexe (type Kanban), notre exécution s'est appuyée sur le diagramme de Gantt qui fixait nos jalons principaux. Le projet a été découpé en macro-tâches, et chaque membre du groupe piochait la mission suivante en flux continu, en fonction des priorités du Gantt et de ses compétences techniques. La plateforme Discord a été désignée comme notre canal de communication officiel pour le projet, permettant des échanges rapides et le partage de code ou de ressources. L'utilisation de Git a été encadrée par des règles de communication strictes. Pour éviter le chevauchement de code sur GitHub et limiter les conflits de fusion, deux pratiques ont été adoptées : prévenir systématiquement le groupe sur Discord lors de la modification d'un fichier critique, ou créer des branches de développement isolées pour les nouvelles fonctionnalités avant de les intégrer au projet principal.

2. Phase de Réalisation

L'exécution technique du projet a reposé sur une philosophie full-stack collective. Plutôt que de cloisonner strictement les rôles dès le départ, nous avons fait le choix que chaque membre de l'équipe touche à l'ensemble du code. Cette approche a permis à tous d'acquérir une compréhension globale de l'architecture. Néanmoins, au fil de l'avancement, des affinités naturelles se sont dessinées, laissant émerger des experts sur certaines technologies précises vers la fin du projet pour peaufiner les détails. La phase la plus critique fut la connexion entre le Front-End et le Back-End. Pour assurer la fluidité de cette intégration, nous avons fonctionné en binôme informel : lorsqu'un membre terminait la création d'une route API ou d'une requête complexe côté Back-End, il sollicitait un autre membre pour développer immédiatement l'affichage Front-End correspondant. Cette communication constante a permis de vérifier instantanément que les requêtes vers la base de données renvoient les bons formats et que l'interface réagit correctement.

Le défi majeur de notre projet n'a pas été la résolution d'un simple bug, mais une tentative de pivot architectural décidé en cours de route. Conscient des limites d'une approche web traditionnelle, nous avons pris la décision de migrer l'intégralité de notre stack Front-End vers ReactJS, propulsé par l'outil de build Vite, et stylisé avec le framework utilitaire TailwindCSS. Ce changement radical a impliqué de repenser totalement la communication client-serveur, nécessitant l'utilisation de requêtes asynchrones (API Fetch) pour request à notre serveur Spring Boot. La courbe d'apprentissage associée à ce pivot est vertigineuse, entraînant des blocages liés à la gestion des états, aux cycles de vie des composants, et aux erreurs CORS. Malgré nos efforts pour surmonter ces obstacles par l'entraide et la documentation, nous avons dû nous rendre à l'évidence : cette transition compromettait sérieusement la livraison du projet. Faisant preuve de pragmatisme, nous avons donc décidé d'effectuer un retour en arrière assumé et de repasser sur notre stack initial.

Afin de garantir la robustesse de CYStation, la phase de validation a été menée avec une grande rigueur, centrée sur la spécificité principale de notre cahier des charges : la hiérarchie des rôles basée sur les points d'expérience. Nous avons généré de multiples comptes utilisateurs fictifs en base de données, en leur attribuant des soldes d'XP artificiels correspondant à chaque pallier critique. À partir de ces comptes, nous avons simulé exhaustivement tous les scénarios d'usage possibles : tentatives d'accès à des zones restreintes, vérification de l'incrémement de l'XP après une action, et accès au panneau d'administration. Cette batterie de tests systématiques nous a permis d'affiner la sécurité des routes de l'API et de corriger les failles d'affichage. Enfin, nous avons validé l'ergonomie de l'interface en utilisant les outils de développement (F12) pour simuler un affichage mobile, garantissant ainsi la réactivité (responsive design) de la station sur différents terminaux. À l'issue de cette phase, l'application finale présentée correspond en tout point à nos attentes initiales et aux exigences du projet. À ce jour, et suite à nos multiples vérifications croisées, aucun bug visible ne subsiste, témoignant de la stabilité de l'application et de la réussite de notre pivot technologique.

3. Résultats obtenus

Le livrable final de ce projet est l'application web CYStation, un hyperviseur fonctionnel développé avec le framework Spring Boot et une base de données MariaDB. Un moteur de simulation hybride intégré, capable de modéliser les cycles de vie de la station. Le produit livré comprend une architecture client-serveur opérationnelle avec une interface utilisateur réactive validée pour les téléphones (HTML/CSS/JS/Thymeleaf) permettant le contrôle des équipements en temps réel via des WebSockets. Un système d'authentification sécurisé (Spring Security) protégeant les données et les accès critiques selon les rôles utilisateurs (XP)

L'objectif de créer un site interactif est atteint. La particularité technique réside dans le modèle de simulation : nous avons implémenté une logique stochastique (aléatoire) pour la production d'énergie et les relevés de capteurs (PPM, radiations), couplée à une logique déterministe pour la dépendance énergétique. Dans le Device Generator la production fluctue entre 80 et 100 unités de manière aléatoire via `Math.random()`. Cependant, si la somme des consommations des équipements dépasse cette production, le système bascule automatiquement les objets en `DeviceStatus.OFFLINE`. Ce mécanisme respecte scrupuleusement les enjeux de survie fixés dans le cahier des charges initial.



Arial

Aa

ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz

CYSTATION
CYSTATION
CYSTATION
CYSTATION

#46CFB3

#000924

#ABFFD1

#041B38

IV. Conclusion et perspectives

1. Bilan global

À l'origine de ce projet, notre équipe s'était fixé pour mission d'appliquer les concepts de l'Internet des Objets à un environnement de survie extrême : une station spatiale. La problématique initiale exigeait la conception d'un système centralisé capable de remonter des données critiques tout en gérant de manière stricte les habilitations d'un équipage. Notre réponse, la plateforme CYStation, remplit ce contrat en offrant un tableau de bord fonctionnel où la hiérarchie et l'accès aux équipements sont dictés par un système d'évolution basé sur des points d'expérience.

L'évaluation de nos objectifs est globalement très positive. Nous avons réussi à livrer une application web robuste et entièrement fonctionnelle en environnement local. Le livrable final a validé avec succès l'ensemble des scénarios d'usage critiques, de l'interaction basique d'un visiteur jusqu'à la gestion des paramètres vitaux par l'administrateur. L'architecture back-end sous Spring Boot et la base de données MariaDB assurent une intégrité parfaite des données. Notre plus grande réussite réside sans doute dans l'optimisation de notre interface mobile. Malgré les défis techniques rencontrés, nous avons su concevoir un CSS adapté aux smartphones, garantissant une expérience utilisateur fluide, réactive et visuellement cohérente pour la maintenance.

Ce projet constitue un retour d'expérience important pour les cinq membres de notre groupe. Sur le plan technique, la progression a été particulièrement marquante. Partant de connaissances en base de données relationnelles mais de connaissances quasi nulles sur les frameworks web, nous avons acquis de solides compétences en développement "full-stack" : de la structuration des entités en Java à la manipulation dynamique du DOM. Sur le plan humain et organisationnel, ce projet est une leçon de gestion et de collaboration. Nous avons appris à coordonner nos efforts via Git en instaurant des règles strictes pour éviter les conflits, à répartir la charge de travail de manière fluide, et surtout à prendre des décisions critiques en équipe pour préserver l'avancement du projet. Cette expérience a considérablement renforcé notre cohésion et notre compréhension des réalités de l'ingénierie logicielle.

2. Pistes d'amélioration

Dans l'hypothèse du développement d'une version 2.0, notre priorité technique immédiate se concentrerait sur l'enrichissement de l'interface utilisateur. Bien que notre intégration HTML/CSS actuelle soit fonctionnelle et réactive, une refonte visuelle plus ambitieuse permettrait de renforcer considérablement l'immersion. L'ajout de nouvelles options d'interaction et une structuration CSS plus complexe offriraient un rendu irréprochable, digne d'un tableau de bord aérospatial de pointe.

Cette refonte graphique serait couplée à l'intégration d'outils d'analyse visuelle des données. La génération de graphiques dynamiques permettrait de suivre l'évolution temporelle des métriques vitales (oxygène, énergie, température). Ces outils d'analyse statistique sont indispensables à la prise de décision dans un tel environnement. L'injection de véritables jeux de données continus viendrait densifier l'expérience, offrant aux utilisateurs de rang Scientifique ou Administrateur de réels outils d'analyse statistique, indispensables à la prise de décision dans un environnement critique.

À plus grande échelle, l'architecture logicielle de CYStation possède le potentiel pour s'affranchir du cadre strictement académique et d'un hébergement local. L'évolution naturelle du projet consisterait dans un premier temps à déployer l'application sur le Cloud, la rendant ainsi accessible à un public externe. À plus long terme, la véritable mise à l'échelle consisterait à relier notre Back-End à des flux de données réels via des API externes, voire à l'interconnecter avec de véritables capteurs physiques. Le projet dépasserait alors le stade de la simulation pure pour se transformer en un véritable hyperviseur IoT, capable de modéliser et de gérer les contraintes d'une véritable Station Spatiale Internationale.